**Faculty Of Engineering and Technology**

**Department of Electrical and Computer Engineering**

**MACHINE LEARNING AND DATA SCIENCE– ENCS5341**

**Assignment #3 Report**

# Multimodal Classification of Travel Images: Predicting Time of Day, Weather, Activity from Images and Descriptions

---

**Prepared By:**

**STD#1:** Abdulrahman Sawalmeh -1221574**.**

**STD#2:** Saleh Kiswani - 1220641.

**STD#3:** Laith Ghoneim - 1221123.

**Instructor:** Yazan Abu Farha.

**Submission Date:** 13/1/2026

# Table of Contents

**Table of Figures** ..... ii

**1. Introduction** ..... 1

**2. Data Preprocessing and Exploratory Data Analysis (EDA)** ..... 1

**2.1. Data Preprocessing** ..... **1**

**2.2. Exploratory Data Analysis (EDA)** ..... **2**

**3. Experiments and Results** ..... 3

**3.1. Model #1: Baseline Model (KNN)** ..... **3**

**3.2. Model #2: SVM** ..... **4**

*3.2.1. Hyper Parameters:* ..... 4

**3.3. Model #3: Logistic Regression** ..... **4**

*3.3.1. Hyper Parameters:* ..... 4

**4. Performance Analysis** ..... 5

**4.1. Confusion Matrices for Best Algorithms Per Target:** ..... **6**

**4.2. Misclassified Examples & Analysis:** ..... 7

**5. Conclusions and Discussion** ..... 8

## Table of Figures

Figure 1: Top 20 Words in the Descrption and the Description Length. 2
Figure 2: Target Label Distribution plots. 3
Figure 3: KNN F1 Score & Accuracy results for (K=1 Vs K=3). 3
Figure 4: Weather (Linear SVC) Confusion Matrix. 6
Figure 5: Time Of Day (Log Regression) Confusion Matrix. 6
Figure 6: Activity (Log Regression) Confusion Matrix. 7
Figure 7: Misclassified Activity Examples. 7
Figure 8: Misclassified Time Of Day Examples. 7
Figure 9: Misclassified Weather Examples. 8

# 1. Introduction

In this project, we focused on predicting **Activity**, **Time of Day**, and **Weather** from the extracted image feature vectors. We selected these three targets because they had the **cleanest labels in the dataset** and gave us the **best performance** compared to other options. To solve the problem, we experimented with three classification models: **K-Nearest Neighbors (KNN)** as a simple baseline, **Logistic Regression** as a strong linear model, and **Support Vector Machine (SVM)** as a more powerful classifier for high-dimensional feature spaces. We evaluated all models using **Accuracy** and **F1-score**. Accuracy measures the overall percentage of correct predictions, while F1-score is especially useful when classes are imbalanced because it combines **Precision** and **Recall** into one value (Precision shows how many predicted positives are correct, and Recall shows how many true positives were successfully found).

# 2. Data Preprocessing and Exploratory Data Analysis (EDA)

## 2.1. Data Preprocessing

We picked the task of predicting **Time of Day, Weather, and Activity** because these are things you can usually tell just by looking at an image and reading a short description. Using **CLIP image text embeddings** make a lot of sense here, since CLIP learns how images and text match each other, so it can capture visual clues like lighting, sky, people, and also what is written in the description.

The original dataset had **1084 samples**, but the labels were not clean at all. There were spelling mistakes, random capital letters, extra spaces, weird symbols, and sometimes people wrote long free-text instead of using clear labels. So before training any model, we had to clean and standardize everything.

First, all text columns were normalized:

- extra spaces, tabs, and punctuation were removed
- broken characters and weird encodings were fixed
- different spellings of the same thing were unified

After that, the labels were mapped to the categories required by the assignment:

- **Activity** → Sightseeing, Relaxing, Hiking, Religious, Adventure/Sports, Water Activities
- **Weather** → Sunny, Cloudy, Rainy, Snowy, Not Clear
- **Time of Day** → Morning, Afternoon, Evening
- **Season** and **Mood/Emotion** were also mapped to their own fixed sets
- **Country** was only cleaned (typos and name variants fixed), not mapped to categories

If a label was unclear or didn't really fit anywhere, we didn't force it we just left it missing. Also, no rows were removed, so we didn't lose any data.

## 2.2. Exploratory Data Analysis (EDA)

After cleaning, we looked at how the data is distributed to get a better idea of what the models will see. One thing that becomes obvious really fast is that the data is **not balanced**.

**Some things that stand out:**

- Most samples are **Sightseeing** or **Relaxing**
- **Sunny** is by far the most common weather
- **Afternoon** appears more than Morning or Evening
- **Summer** is much more frequent than Winter
- The text descriptions are usually pretty short, around **20 words** on average

Because of this imbalance, accuracy alone can be misleading, so we also use **Macro-F1** when we evaluate the models.

**Figures**

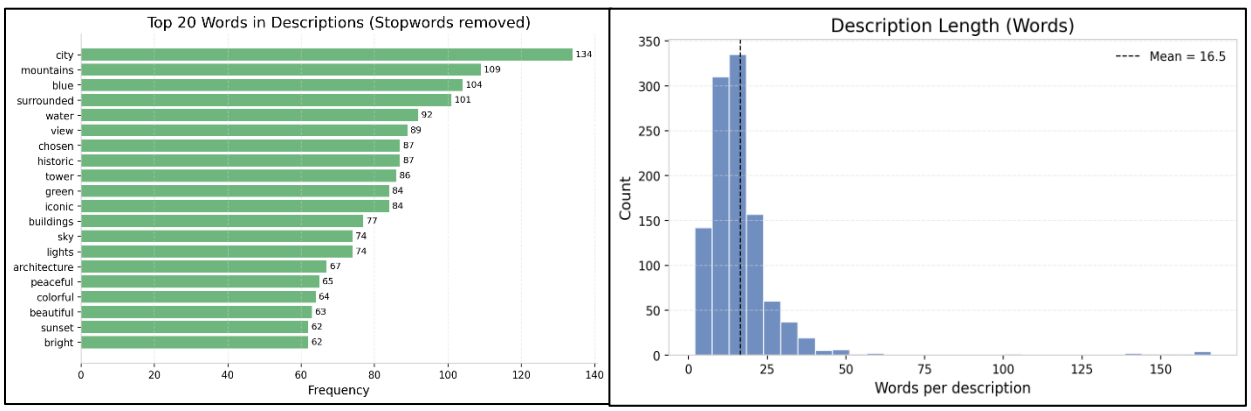


*Figure 1: Top 20 Words in the Descrption and the Description Length.*

The description-length histogram shows most captions are short (mean ≈ 16.5 words) with a long right tail, and the top-words plot highlights common travel/scene terms (e.g., *city, mountains, water, sky*), suggesting consistent descriptive content.

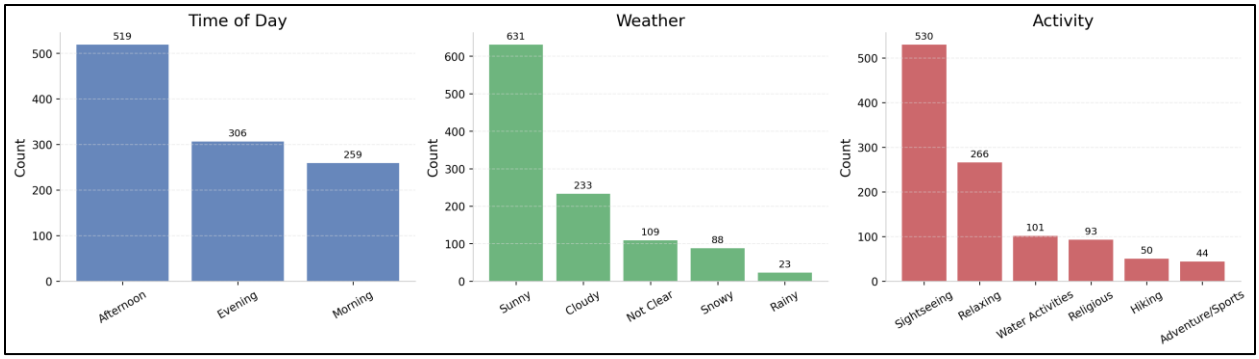


*Figure 2: Target Label Distribution plots.*

The label distribution plots reveal noticeable class imbalance especially Sunny weather and Sightseeing activity dominating which can bias models and makes macro-F1 and class weighting/stratified splits important.

# 3. Experiments and Results

## 3.1. Model #1: Baseline Model (KNN)

For our baseline, we went with KNN because it's simple and gives us a good starting point to compare everything else with. We loaded the labels from *Data_Cleaned_v3.csv* and the image feature vectors from *X_fused_clip.npz*, then we tried to predict Activity, Weather, and Time of Day. For each target, we removed the rows with missing labels, then split the data into 80% train and 20% test *(random_state=42)* and used stratify so the class balance stays basically the same in train and test.

After that, we trained KNN with cosine distance, and we tuned the main hyperparameter which is k (we tested k=1 and k=3). We evaluated using Accuracy and Macro F1-score, and we also saved normalized confusion matrices to see what classes are getting mixed up. Then for the proposed models, we trained Logistic Regression and SVM on the exact same splits/features so it's a fair comparison. For tuning, we mainly adjusted C for both Logistic Regression and SVM (and for SVM we also tried different kernels if needed), and we kept the best settings based on validation performance before reporting the final test results.

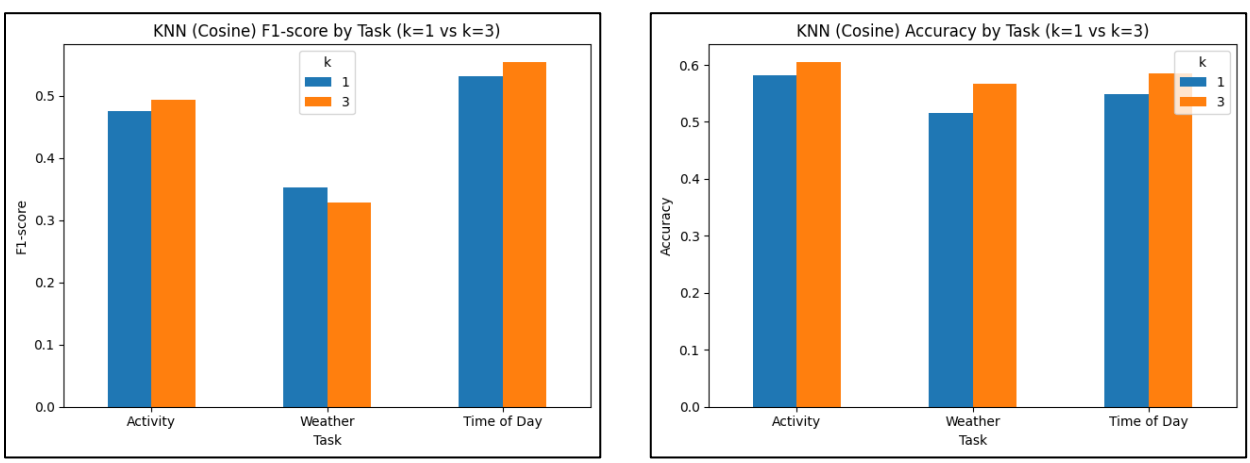


*Figure 3: KNN F1 Score & Accuracy results for (K=1 Vs K=3).*

### 3.2. Model #2: SVM

We chose SVM because it is known to perform well with **high-dimensional input features**, especially when those features are embedding extracted from images. SVM also has good generalization ability, and it usually produces strong results in classification tasks when the dataset is not perfectly clean.

We compared the SVM results with the KNN baseline (k = 1 and k = 3). For some targets, SVM performed better because it learns a more general decision boundary instead of relying only on the nearest neighbors. However, in other cases the improvement was limited, mainly because some classes look very similar in images and the dataset is also imbalanced, with certain classes appearing much more than others.

#### *3.2.1. Hyper Parameters:*

In LinearSVC (linear SVM), the main tuned parameter is C, which controls the soft-margin/regularization tradeoff (smaller C ⇒ wider margin & more regularization; larger C ⇒ tighter fit & less regularization). In our code we tuned *C = [0.01, 0.1, 1, 10, 100]* and used *class_weight="balanced"* to reduce bias from class imbalance (no kernel/gamma since it's linear).

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

- C (regularization / margin penalty): tested as *[0.01, 0.1, 1, 10, 100].*

### 3.3. Model #3: Logistic Regression

We chose Logistic Regression because it is a fast and effective linear classifier, and it is often used as a strong baseline for comparison in classification tasks. It works especially well when the classes can be separated with a roughly linear boundary in the embedding space.

We compared Logistic Regression with the KNN baseline. In some cases, it can achieve better results because it learns in a more stable and consistent way. However, if the dataset is highly imbalanced or the classes are not visually distinct, the improvement may still be limited.

#### *3.3.1. Hyper Parameters:*

In Logistic Regression, **C** is the inverse regularization strength: **smaller C** applies stronger L2 regularization (simpler weights, less overfitting), while **larger C** weakens regularization (more flexible fit). In our tuning we tested ***C ∈ {0.001, 0.01, 0.1, 1, 10, 100, 1000}*** to find the best tradeoff between generalization and fitting the training data (optimized for macro-F1).

$$\hat{W} = \arg\min_{W}\left(-\sum_{i=1}^{n}\log p(y_i \mid x_i; W) + \frac{1}{2C}\|W\|_2^2\right)$$

| Target | Model | Hyperparameters | Accuracy | Macro-F1 |
|---|---|---|---|---|
| **Activity** | kNN (k=1) | — | 0.5822 | 0.4754 |
| **Activity** | kNN (k=3) | — | 0.6056 | 0.4946 |
| **Activity** | LogReg (best C) | C = 100 | 0.6590 | 0.6208 |
| **Activity** | LinearSVC (best C) | C = 10 | 0.6544 | 0.6159 |
| **Weather** | kNN (k=1) | — | 0.5161 | 0.3534 |
| **Weather** | kNN (k=3) | — | 0.5668 | 0.3284 |
| **Weather** | LogReg (best C) | C = 50 | 0.6406 | 0.5610 |
| **Weather** | LinearSVC (best C) | C = 1 | 0.7005 | 0.6127 |
| **Time of Day** | kNN (k=1) | — | 0.5484 | 0.5325 |
| **Time of Day** | kNN (k=3) | — | 0.5853 | 0.5551 |
| **Time of Day** | LogReg (best C) | C = 5 | 0.6498 | 0.6443 |
| **Time of Day** | LinearSVC (best C) | C = 1 | 0.6359 | 0.6094 |

*Table 1: Final Best Results & Performance Comparison for all models.*

## 4. Performance Analysis

Across all three targets, the learned linear models clearly **outperform** the kNN baseline: for Activity, performance rises from kNN (k=3) Acc=0.6056 / Macro-F1=0.4946 to LogReg (C=100) Acc=0.6590 / Macro-F1=0.6208, with LinearSVC (C=10) very close (0.6544 / 0.6159). For Weather, the best model is LinearSVC (C=1) achieving Acc=0.7005 / Macro-F1=0.6127, beating both LogReg (C=50) 0.6406 / 0.5610 and kNN (Macro-F1 $\leq$ 0.3534). For Time of Day, LogReg (C=5) performs best (Acc=0.6498 / Macro-F1=0.6443), while LinearSVC (C=1) is lower (0.6359 / 0.6094) and kNN remains behind (k=3: 0.5853 / 0.5551). Overall, the results suggest that linear decision boundaries on CLIP fused embeddings generalize better than instance-based kNN, and the best choices are LogReg for Activity/Time of Day and LinearSVC for Weather.

## 4.1. Confusion Matrices for Best Algorithms Per Target:

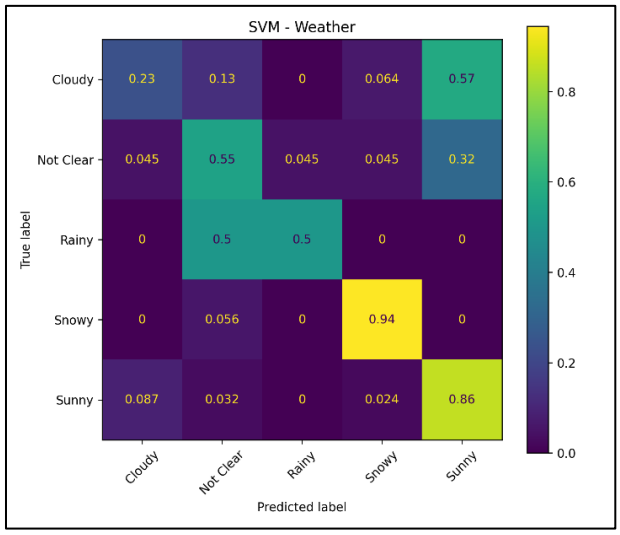


*Figure 4: Weather (Linear SVC) Confusion Matrix.*

**Weather (SVM):** Very strong recognition of *Snowy (~0.94)* and *Sunny (~0.86)*, but *Cloudy* is often predicted as *Sunny (~0.57)* and *Rainy* splits heavily into *Not Clear (~0.5)* and *Rainy (~0.5)*, indicating ambiguity between overcast/low-visibility conditions.

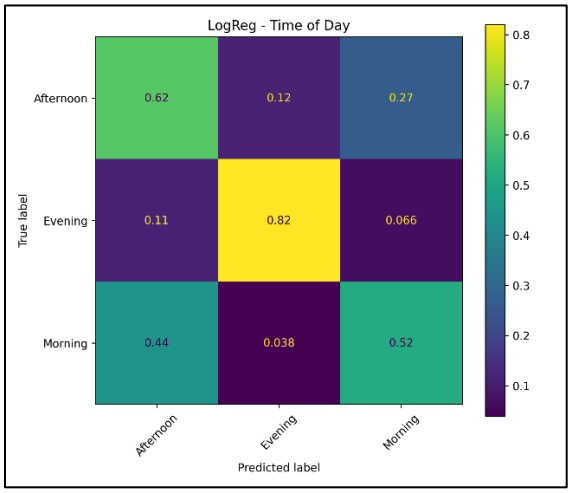


*Figure 5: Time Of Day (Log Regression) Confusion Matrix.*

**Time of Day (LogReg):** *Evening* is easiest (**~0.82** correct), but *Morning* and *Afternoon* are frequently confused (e.g., *Morning* → *Afternoon ~0.44* and *Afternoon* → *Morning ~0.27*), which makes sense because lighting cues can be subtle around boundary times.

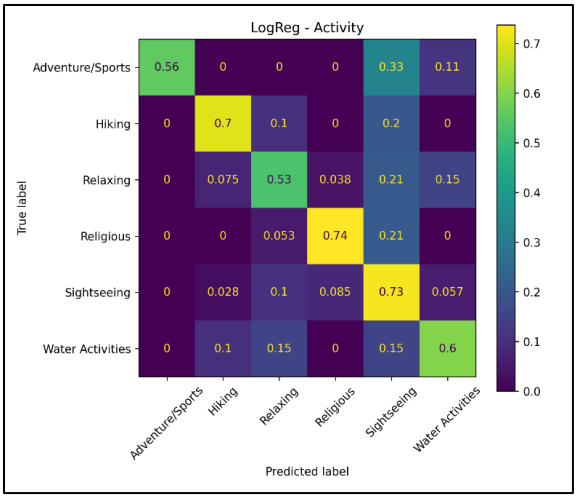


*Figure 6: Activity (Log Regression) Confusion Matrix.*

**Activity (LogReg):** Strong diagonal performance for *Religious (~0.74)* and *Sightseeing (~0.73)*, while the biggest confusion is *Adventure/Sports → Sightseeing (~0.33)* and some mixing among *Relaxing/Water Activities/Sightseeing*, showing overlap between visually similar leisure scenes.

## 4.2. Misclassified Examples & Analysis:

*Figure 7: Misclassified Activity Examples.*

*Figure 8: Misclassified Time Of Day Examples.*

*Figure 9: Misclassified Weather Examples.*

**Weather:** The mistakes are mostly "in-between" scenes aurora/night sky getting predicted as Snowy, rainy streets as Not Clear, and Cloudy → Sunny when the lighting is super bright.

**Time of Day:** It mainly mixes up Morning vs Afternoon because daylight looks really similar in a lot of photos.

**Activity:** Confusions are between similar-looking stuff, like Adventure/Sports → Water Activities, Sightseeing → Water Activities, and sometimes Water Activities → Hiking for mountain-lake scenes.

## 5. Conclusions and Discussion

In this project, we built an end-to-end machine learning pipeline to predict Weather, Time of Day, and Activity from a travel dataset using fused CLIP image and text embeddings. We first explored the data through EDA to understand label distributions, text characteristics, and sources of imbalance, then extracted feature representations using the ViT-L/14 CLIP model. For modeling, we established a baseline with kNN (k=1 and k=3) and compared it against stronger classifiers, mainly Logistic Regression and Linear SVM, with hyperparameter tuning to improve generalization. The experiments showed that tuned linear models consistently outperformed the kNN baseline, with Logistic Regression performing best for Activity and Time of Day, and LinearSVC performing best for Weather. Finally, our error analysis (confusion matrices and misclassified examples) highlighted that most remaining mistakes are due to class imbalance and label overlap, where visually similar conditions (e.g., Cloudy vs Sunny) or ambiguous scenes make the target hard to distinguish. Overall, the results confirm that CLIP-based fused embeddings provide a strong foundation for multi-class prediction, while performance is still bounded by dataset balance and the inherent ambiguity of some labels.